

Assessment 3: Individual Contribution

Mikhail Zagoruiko (A00163187)
Design and Creative Technologies, Torrens University
PBT205: Project Based Learning Studio: Technology
Fahad Hameed
May 9, 2026

Body

The contact tracing prototype was my solo work across the semester. I wrote the Rust backend, the tracker service, the Vite/TypeScript front end, the Docker Compose setup, the RabbitMQ wiring, the Postgres schema and migrations, local credential flows, and the build and deployment scripts. The architecture presentation that accompanied Assessment 2 was also mine.

The piece I am most pleased with is the position synchronisation flow that lets multiple clients see each other moving across a shared grid in real time. The system uses several RabbitMQ channels rather than one, with presence updates and contact recording each running on their own, which keeps each path simple to reason about and let the tracker service slot in cleanly without disturbing the movement or chat code. It is the part of the architecture I am most likely to reuse in future work.

Getting Docker Compose, Postgres, and the RabbitMQ flow to coexist in the same fortnight in February was the moment when the prototype stopped being a handful of separate experiments and started being one system. I had used Docker and Postgres before but never together under a deadline, and the experience taught me more about integration than the rest of the semester combined.

The post-prototype work in April and May was also mine. The integration test on 13 April was the first piece of automated coverage on the message-passing flow; the Vite upgrade on 29 April brought the front-end tooling back in line with the current ecosystem; the adaptive UI on 6 May gave the prototype a usable layout on phones and tablets as well as desktops, which closed the gap between a prototype that ran on my laptop and one that could be shown on any device.

The mistake I made was about pacing. I treated the "Final commit" at the end of March as the end of the project, which it was not, and the April and May work was done under more time pressure than it needed to be. The integration test in particular should have been written alongside the position synchronisation feature in February rather than two months later; writing it earlier would have caught a small ordering issue in the tracker that I instead caught by hand in March.

What I take from the project is the value of integrating early and being explicit about scope cuts. Both are habits, not techniques. Integrating early meant the seams between Postgres, RabbitMQ and the front end were already familiar by the time later features needed them. Being explicit about scope cuts kept the prototype honest about what it was and was not, which made the process easier.